

Quiz 7B

Filename: q7.c

Instructions

Use the provided starter code, which you can access [here](#), and copy it in its entirety. This file is intentionally incomplete to simulate a written exam. You are strongly encouraged to first work out your solutions using pencil and paper. Once you are confident in your answers, type the final solutions into this file. You may only write code inside the bodies of the required functions. Do not modify function signatures, `#include` directives, or any other part of the file. Write your answers using a plain text editor only. Do not use a compiler, debugger, or IDE while completing this file. Save your file as `q7.c`. You may submit multiple times on Gradescope before the submission deadline. Only the final submission made before the deadline will be graded.

Important: Certain compiler warnings are treated as errors, and any such warning will cause compilation to fail. If your code does not compile on Gradescope, your submission will receive a score of 0. All successfully compiled submissions will be evaluated for both correctness and proper implementation.

Exam Preparation Tip

As you work through the problems, you must time yourself. Each question is designed to be completed within a reasonable amount of time. No question should take more than 10 minutes.

TODO 1 (50 points)

Write a function `heapInsert()` that inserts a given item into a min-heap. The function should accept the following parameters: the heap array, a pointer to the current number of elements in the heap, the array's capacity, and the item to be inserted. Assume that the root of the heap is stored at index 0.

If the heap does not have enough space, it returns 0. If the insertion is successful, it returns 1.

Assume the following helper functions are already implemented:

```
void percolateUp(int arr[], int arrSize, int upIndex);  
// Performs a percolate-up operation starting at index upIndex.  
// Note: arrSize refers to the number of items (i.e., heapSize).  
  
void percolateDown(int arr[], int arrSize, int downIndex);  
// Performs a percolate-down operation starting at index downIndex.  
// Note: arrSize refers to the number of items (i.e., heapSize).
```

Important Note

Percolate-up corresponds to the operation referred to in class as heapify-up, while percolate-down corresponds to heapify-down.

```
int heapInsert(int heapArr[], int *heapSize, int heapCapacity, int item) {
```

```
}
```

TODO 2 (50 points)

Consider the following definition of a trie node:

```
typedef struct TrieNode_s {  
    struct TrieNode_s *children[26];  
    int is_end_of_word;  
} TrieNode;
```

Write a function `hasPrefixPlusOne` that returns 1 if there exists a word in the trie that is exactly one character longer than a given prefix, and returns 0 otherwise.

In other words, determine whether there exists a word of the form:

prefix + one additional character

Example

If the trie contains the words `cat`, `car`, and `cart`, then:

- `hasPrefixPlusOne(root, "ca")` returns 1, since both `cat` and `car` match the pattern.
- `hasPrefixPlusOne(root, "cat")` returns 0, since there is no word of length 4 that begins with `"cat"`.

You may assume that if a child pointer in the trie is not `NULL`, then there exists at least one word in that subtree. Further, you may assume that all words consist exclusively of lowercase English letters and that `prefix` is a null-terminated string.

```
int hasPrefixPlusOne(TrieNode *root, char *prefix) {
```

```
}
```